

LAB PRACTICUM: OpenMP Multi-Core Scaling in Python — Completed Report

Student Name:	Marzhan Sherekhan	Student ID:	230109015
Hardware / CPU:	12th Gen Intel Core i7-12700H — 14 cores (6 P-cores + 8 E-cores), 20 threads; Windows; Python 3.14, NumPy 2.4.2, Numba 0.65.1	Date & Section:	02.10.2026 / ____

1. Environment Setup

Packages installed with `pip install numpy numba matplotlib` (numpy 2.4.2, numba 0.65.1, matplotlib 3.10.9). `numba.config.NUMBA_NUM_THREADS` reported **20** hardware threads, so the sweep was 1, 2, 4, 8 and 20 threads. Laptop on wall power, background apps closed.

2. Challenge 1 — Core Speedometer & Amdahl's Law

Threads	Time (s)	Speedup	Efficiency (%)	π estimate
1	0.8021	1.00x	100.0	3.141597
2	0.4048	1.98x	99.1	3.141821
4	0.2172	3.69x	92.3	3.141661
8	0.1651	4.86x	60.7	3.141706
20	0.1294	6.20x	31.0	3.141417

A. Baseline T_1 and $T_{\max}$

$T_1 = 0.8021$ s (1 thread); $T_{\max} = 0.1294$ s (20 threads). Best speedup = **6.20x**, efficiency at 20 threads = 31.0%.

B. Why is the warm-up call required?

@njit compiles lazily: the first call runs type inference, LLVM optimisation and machine-code generation, which takes far longer than the 0.13–0.8 s runs measured here. It also starts the threading layer and its worker thread pool. Calling it once on a tiny input caches the compiled code, so the timed runs measure only execution, not compilation or start-up.

C. Did efficiency stay at 100%?

No. It stayed near ideal up to 2 threads (99.1%) and 4 threads (92.3%), then fell sharply to 60.7% at 8 and 31.0% at 20 threads. My i7-12700H does not have 20 equal cores. It has 6 fast performance cores (P-cores) with Hyper-Threading (12 threads) plus 8 slower efficiency cores (E-cores, no Hyper-Threading), for 14 cores / 20 threads in total. (1) **SMT/Hyper-Threading**: 20 threads run on only 14 physical cores. On the 6 P-cores, two logical threads share one core's ALUs/FPUs, so the second thread adds little for this compute-bound loop. (2) **Hybrid P/E cores**: prange splits the work into roughly equal chunks, so chunks on the slower E-cores finish last while the P-cores wait. Up to 4 threads can all run on P-cores (92% efficiency); at 8 and 20 threads E-cores and shared Hyper-Threaded cores are used, and efficiency drops to 61% and 31%. (3) **Turbo and thermal limits**: one active core runs at maximum boost; with all cores active, power limits lower the clock speed of every core. (4) **Fixed overhead**: at 0.13 s total, thread wake-up, scheduling and the final reduction take up a larger share of the time (Amdahl's Law: $S = 1/((1-p)+p/N)$). Shared caches and the memory bus add some contention too, but with no large arrays this effect is minor.

D. Why does `inside_circle += 1` inside prange not corrupt the count?

Numba sees a scalar updated only with `+=` inside the prange body and turns it into a **reduction variable**. Each thread gets a private copy starting at 0, counts its own chunk without sharing, and the partial sums

are added together once at the end. This is the same as OpenMP `#pragma omp parallel for reduction(+:inside_circle)`, so there is no race condition and no lock or atomic operation in the hot loop.

3. Challenge 2 — Load Imbalance & Dynamic Scheduling

Decomposition (20 threads)	Time (s)
Row-parallel (prange over rows)	0.547
Column-parallel (prange over columns)	0.545

A. Which decomposition was faster and why?

On my laptop the two were effectively tied: rows 0.547 s, columns 0.545 s (a 0.4% difference, within run-to-run noise). Memory layout barely matters here because the kernel is **compute-bound**. Each pixel needs up to 1000 iterations of register-only arithmetic but only one 4-byte store at the end, so the cost of a cache-unfriendly store is hidden by the computation. A modern laptop also has large per-core L2 and shared L3 caches that absorb the strided writes. Both versions also give a similar load balance, because the set is roughly symmetric along both axes. Row-parallel is still the safer choice, since it writes contiguous memory and avoids false sharing between threads.

B. C-contiguous (row-major) layout and cache lines

NumPy stores a 2-D array row by row, so `img[r, c]` and `img[r, c+1]` are next to each other in memory. A 64-byte cache line holds 16 int32 values. In the row version the inner loop runs over `c`, so writes are sequential and each loaded line is fully used. Each thread also owns whole rows, so no two cores write to the same line. In the column version the inner loop runs over `r`, so each write jumps $W \times 4 = 10,000$ bytes to a new cache line, using only 4 of its 64 bytes. Threads handling neighbouring columns also write to the same cache lines, which forces those lines to bounce between cores (false sharing / coherence traffic). This costs more in memory-heavy kernels; in this compute-heavy fractal it was hidden (see A).

C. Load imbalance and dynamic scheduling

Work per pixel depends on how quickly it escapes. Top rows ($\text{Im} \approx -1.2$) lie mostly outside the set and escape in 2–3 iterations, so thread 0 finishes fast and then sits idle. Centre rows cross the main cardioid, where many pixels run all 1000 iterations, so thread 2 has far more work. With static scheduling the total time equals the slowest thread's time. OpenMP `schedule(dynamic, chunk)` (or guided) splits the loop into small chunks kept in a shared queue. A thread that finishes takes the next chunk, so fast threads keep working until all rows are done and no core stays idle. In Numba, the `TBB/workqueue` backends and `numba.set_parallel_chunksize()` give the same kind of balancing.

D. Exported image

`mandelbrot_output.png`, generated on my laptop by `ch2.py` (2500×2500, saved at 300 dpi), is attached with this report.

4. Challenge 3 — Stencil Computation & Memory Bandwidth

Precision	Threads	Time (s)	Throughput (Mcells/s)
float64	20 (max)	0.125	5412.63
float32	20 (max)	0.313	2154.98

A. Runtime and throughput

float64, 20 threads: **0.125 s, 5412.63 Megacells/sec** ($1500 \times 1500 \times 300 = 675$ M cell updates).

B. float64 → float32

Runtime did **not** decrease. It rose from 0.125 s to 0.313 s, a factor of 0.40x (2.5x slower). Throughput fell from 5413 to 2155 Mcells/s. The cause is in the code, not the hardware: `alpha=0.20` and the literal `4.0`

are Python float64 values. Numba therefore promotes every float32 neighbour to float64, computes in float64, and converts back to float32 on each store. These extra conversions per cell block clean SIMD vectorisation, so the kernel is now limited by instructions rather than by memory. In theory a stencil is memory-bandwidth bound, since each update does only ~6 flops but loads 5 neighbours and stores 1 value. Halving the bytes per cell (4 vs 8) should help much more than extra compute power: twice as many cells fit per cache line and per SIMD register, and the two grids shrink from 36 MB to 18 MB, closer to fitting in L3 cache. To get that benefit, alpha and the constants must also be float32 (e.g. `alpha=np.float32(0.2), np.float32(4.0)`), which keeps the whole loop in single precision.

C. Memory roofline — why 2x cores ≠ 2x speedup

The roofline model gives attainable performance = min(peak FLOP/s, arithmetic intensity × memory bandwidth). The stencil's arithmetic intensity is very low (about 6 flops per ~16 bytes of DRAM traffic for float64, ≈ 0.4 flop/byte), so it sits on the sloped, bandwidth-limited part of the roofline. More cores raise the flat compute ceiling but not the sloped bandwidth line. A laptop has only two memory channels, and a few cores can already saturate them. After that, extra cores just wait for the same DRAM, so speedup flattens. Performance improves by moving fewer bytes (float32 done correctly) or by raising data reuse (cache blocking/tiling, temporal blocking over several timesteps), not by adding threads.

5. Student Result Scorecard (Table 1)

Benchmark Challenge	Active Threads	Array Size / Samples	Time (s)	Speedup	Efficiency (%)
Ch.1 Monte Carlo	1 Thread (Base)	120,000,000	0.8021	1.00x	100.0%
Ch.1 Monte Carlo	2 Threads	120,000,000	0.4048	1.98x	99.1%
Ch.1 Monte Carlo	4 Threads	120,000,000	0.2172	3.69x	92.3%
Ch.1 Monte Carlo	Max Threads (20)	120,000,000	0.1294	6.20x	31.0%
Ch.2 Mandelbrot (Rows)	Max Threads (20)	2500 x 2500	0.547	N/A	N/A
Ch.2 Mandelbrot (Cols)	Max Threads (20)	2500 x 2500	0.545	N/A	N/A
Ch.3 Heat Stencil	Max Threads (20)	1500 x 1500 x 300	0.125 (5412.63 Mcells/s)	N/A	N/A

Additional Ch.1 data point: 8 threads — 0.1651 s, 4.86x, 60.7%.

Whiteboard entry: Intel Core i7-12700H (14 cores / 20 threads) — best speedup **6.20x** (Challenge 1, 20 threads).